ETH
Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

On Combiners for Verifiable Random Functions

Bachelor Thesis

Basil Rohner

March 17, 2026

Advisors: Prof. Dr. A. D. Visor, Dr. P. Ostdoc
Department of Computer Science, ETH Zürich

Acknowledgments

Abstract

Contents

Chapter 1

Introduction

1.1 Motivation and Previous Work

Verifiable Random Functions. A verifiable random function (VRF) introduced by Micali, Rabin and Vadhan in [**?**] is a cryptographic primitive which allows the holder of a secret key to evaluate a pseudorandom function at arbitrary points and obtain image-proof pairs. In the absense of the proof the images are computationally indistinguishable from random elements in the range while the corresponding proof allows the owner of the corresponding public key to verify whether the output was indeed computed correctly. Additionally we require that the verification accepts an image-proof pair if and only if the image is the unique evaluation of the point under the given secret key. As such, a VRF can be considered the public-key equivalent of the more common secret-key pseudorandom function (PRF).

In practice VRFs are used to implement signatures with strong verifiability guarantees [**?**], resettable zero-knowledge proofs [**?**], lottery and e-cash systems [**?**, **?**, **?**] and in Proof-of-Stake blockchains through distributed leader election protocols [**?**, **?**, **?**, **?**].

In addition to standard VRFs, both stronger variants such as constrainable VRFs which additionally allow the constraining of secret keys to a specified subset of evaluations [**?**], key-homomorphic and aggregable VRFs that allow the aggregation of multiple image-proof pairs into a single image-proof pair [**?**] and unbiasable VRFs that guarantee unbiasability, preventing adversaries to generated keys such that the corresponding evaluations can be biased [**?**].

We also consider weaker notions of VRFs, such as verifiable unpredictable functions (VUFs) already introduced in [**?**] together with VRFs and weakly verifiable random functions (wVRFs) which only require a weaker notion of pseudorandomness, that is pseudorandomness on uniformly random evaluation points. While VRFs can be constructed from VUFs using hardcore

bits [**?**], the authors of [**?**] show that VRFs can not, in general, be constructed from wVRFs using a seperation relying on enhanced trapdoor permutations.

Robust (Black-Box) Combiners. Robust combiners are constructions that take many instances of the same cryptographic primitive, called candidate schemes, in form of a description of their Turing machine, and securely implement that cryptographic primitive if a threshhold of the candidate schemes are secure. Since we generally intend for the combiner to derive its security from the subset of secure candidate schemes, and not by implementing a secure instance of the underlying primitive itself and ignoring the candidates, we require black-box reductions from the combiner construction to any sufficiently large subset of its candidates. In this work mainly consider robust black-box combiners, a subclass of combiners that restricts access of the combiner to its candidate schemes to oracles which can be queried but not inspected for their underlying description. Robust combiners allow the hedging of trust in cryptographic hardness assumptions underlying computational security properties, by basing their security on a subset of hard problems rather than a a single hard problem. In practice robust combiners are uesful to reduce the dependency of a systems on the security of a specific third party implementation, e.g. a cryptographic library. In theory robust combiners are related to universal cryptographic construction, i.e. constructions that implement a secure instance of a cryptographic primitive under the only assumption that a secure instance of the cryptographic primitive exists, a prominent example is Levin's universal one-way function[**?**]. Finally combiners play a central role in the transition towards post-quantum cryptography where hardness assumptions are not as well studied as the respective assumptions for the classic cryptography counterpart, in particular the combination of classical and post-quantum cryptographic assumptions has been a fruitful research field.

Combiners have been considered for a wide variety of cryptographic primitivey, including key exchange protocols [**?**], signature schemes [**?**], oblivious-transfer [**?**], zero-knowledge proofs [**?**], commitment schemes [**?**], and indistinguishability obfuscation [**?**]. While pseudorandom functions (PRFs) admit easy robust $(n;1)$ combiners by simply XOR'ing the outputs of the candidate schemes, evaluated on independent keys the combination of VRFs, the combination of VRFs is less obvious. To the best of our knowledge, there has been no prior research on robust combiners for verifiable random functions.

Our Contribution

We introduce a new definition of a robust black-box combiner, a robust black-box DAG combiner, which models the implemented algorithms of

the combiner as computations that follows the structure of a vertex-colored directed acyclic graph starting from a source and ending in a sink, where for each vertex in the graph we query a specific oracle of a specific candidate on the output of a probabilistic polynomial time Turing machine of all predecessors. We show that the notion of a robust black-box DAG combiner is sufficiently strong to capture all ppt. robust black-box combiners. We use this new model to show the impossibility of combining multiple instances of wVRFs which extends to the impossibility of combining VUFs and VRFs without additional assumptions. We argue that the combination of the unique provability property is however less important since it is of information theoretic or statistical nature and therefore doesn't rely on any computational hardness assumption. In fact, good VRF constructions such as ECVRF [**?**], lattice VRF [**?**] and [**?**] mathematically prove unique provability mathematically, rendering combination less important. We consider the possibility to combine more advanced security properties on VRFs such as unbiasability for which we construct a combiner for uniform constructions which relies on collision resistant hash functions and randomness extractors to combine unbiasable VRFs under the additional assumption that all candidate schemnes are uniquely provable. We also investigate combiners for secret-key aggregability and public-key aggregability. Secret-key aggregability is a publicly verifiable property due to an efficient algorithm by Blum, Luby and Rubinfeld [**?**] which allows checks for the homomorphism property of a function in time independent of the size of the underlying groups. Combiners for publiy-key aggregability Finally we investigate additional properties such as constrainability [**?**] nd which all admit the simple pseudorandomness combiner construction.

.

a

1.2 Technical Overview

Robust Black-Box DAG Combiner. We present another specification of robust black-box combiners, termed robust black-box DAG combiners, which describe a form of black-box combiners that are structured by a directed acyclic graph for each implemented algorithm of the underlying cryptographic primitive in **??**. We show that the newly introduced notion in **??** is in fact equivalent to the standard definition of robust black-box combiners **??**, a result we formalize in the following theorem.

restate

Theorem 1.1

Impossibility of Black-Box VRF Combiner. Subsequently we show that there exists no robust $(n;k)$ black-box combiner that takes as input n candidates, of which k securely implement a VRF according to **??** the combiner implements a VF in **??** that is statistically correct according to **??**, uniquely provable according to **??**, and weakly pseudorandom according to **??**. That is

even assuming that $n-1$ candidates implement secure VRFs according to **??** the combiner cannot guarantee to implement a statistically correct weak VRF. We formally state this in the following theorem. restate

Theorem 1.2

Relaxed Combiners for VRF. Having established **??** we move on by assuming that every candidate scheme is uniquely provable, which enables a simple robust $(n;1)$ black-box combiner for VRF in **??** which samples independent keys per candidate generically for the key-generation algorithm, for evaluation the combiner evaluates every candidate on the input with their corresponding secret key, returns the XOR of the images with the proof contiaining all views of the candidate schemes. For verification the combiner verifies that all candidates views in the proof are consistent by querying every candidates verification oracle and then recomputes the output to compare it with the image. The construction allows us to state the following theorem. restate

Theorem 1.3

Relaxed Combiners for Unbiasable VRF. Having relaxed the unique provability property of the candidates we consider combiner constructions for more advanced notions of VRFs. A VRF is fully unbiasable if no efficient adversary can craft a set of keys such that the output distribution distribution is skewed into any direction. We show that fully unbiasable VRF as defined in **??** admit non-uniform robust $(2;1)$ black-box combiners that base their property on collision resistant hash functions and 2-source randomness extractors. . Formally we formulate the following theorem. this is unclear

Theorem 1.4 restate

Relaxed Combiners for Aggregable VRF. Then we consider aggregable VRF we define in **??** which additionally allow the aggregation of many image-proof pairs into a single image-proof pair to realize efficient batch verification. Key-homomorphic, secret-key aggregable, VRF require that the evaluation algorithm forms a group homomorphism with respect to group structures on the secret-key space and the product of the output codomain and the proof space. Using efficient grouphomomorphism tests by [**?**] we can construct a $(n;1)$ robust combiner for statistically key-homomorphic VRF that works by publicly checking the homomorphism property before running the algorithm. restate

Theorem 1.5

For public-key aggregability

Relaxed Combiners for Constrainable VRF. Finally we consider constrainable VRF, defined in **??**. Constrainable VRF additionally allow the user to generate constrained keys, which only allow the evaluation on a constrained set of evaluation points.

Chapter 2

Notation and Preliminaries

2.1 Notation

For a positive integer n we define $[n] = \{1, \ldots, n\}$. Unless specified otherwise we use λ to denote the security parameter. A function $\varepsilon(n)$ is called negligible, if it vanishes faster than the inverse of any polynomial in n, formally defined in **??**. We say that an event E happens with overwhelming probability if $\Pr[E] \geq 1 - \varepsilon(\lambda)$ for a negligible function ε and we say that an E occurs with significant probability if $\Pr[E] \geq \frac{1}{p(\lambda)}$ for a polynomial p in λ. We use $\mathsf{poly}(n)$ to denote the set of functions polynomial in n and $\mathsf{negl}(n)$ to denote the set of functions negligible in n. For a set S we write $y \leftarrow_\$ S$ to indicate that the value of y is sampled uniformly at random from S, for a random variable X we write $y \leftarrow_\$ X$ to indicate that the value of y is sampled from the corresponding sample space with the underlying distribution, and for a randomized algorithm $\mathcal{A}$ we write $y \leftarrow_\$ \mathcal{A}(x)$ to assign to y the value of the output of $\mathcal{A}$ on x. We use $\leftarrow$ for simple assignment. We denote column vectors with lower-case bold letters (e.g. $\mathbf{x}$) . Unless specified otherwise we assume that any algorithm gets as input in the first argument the unary encoding of the security parameter 1^λ. Finally we use ppt. to abbreviate probabilistic polynomial time in context of turing machines and algorithms.

I needed this previously bu

Definition 2.1 (Polynomial Functions) *We say that a function $f : \mathbb{N} \to \mathbb{R}$ is polynomial in its argument λ if*

$$\exists c, d \in \mathbb{N}, \forall \lambda \in \mathbb{N}, f(\lambda) \leq c\lambda^d$$

Definition 2.2 (Negligible Functions) *We say that a function $f : \mathbb{N} \to \mathbb{R}$ is negligible in its argument λ if*

$$\forall c \in \mathbb{N}^+, \exists n_0 \in \mathbb{N}, \forall \lambda > n_0, |f(\lambda)| < \frac{1}{x^c}$$

2.2 Verifiable Functions

A verifiable function (VF) VF is a cryptographic primitive that implements a key-generation algorithm $\mathsf{Gen}_{\mathsf{VF}}$ to sample a key-pair $(\mathsf{sk}, \mathsf{pk})$ containing a secret and a public key, a deterministic evaluation algorithm $\mathsf{Eval}_{\mathsf{VF}}$ that allows the holder of a secret key sk to make evaluations at any points $x \in \{0,1\}^\lambda$ to obtain an image $y \in \mathcal{Y}$ and a corresponding proof $\pi \in \mathcal{P}$, and a verification algorithm that allows a different party with access to the corresponding public key pk to verify that y was correctly computed from x if given access to the proof π. The definition takes inspiration of the definition of a verifiable random function (VRF) as defined by [**?**], to reduce redundand when considering VRFs with additional or relaxed properties we define the properties seperately on top of a verifiable function (VF).

Definition 2.3 (Verifiable Function) *A verifiable function (VF)* VF *is a triple of efficient (ppt.) algorithms* $(\mathsf{Gen}_{\mathsf{VF}}, \mathsf{Eval}_{\mathsf{VF}}, \mathsf{Vfy}_{\mathsf{VF}})$ *with the syntax*

this is less elegant now

- $\mathsf{Gen}_{\mathsf{VF}}$*: The key-generation algorithm takes as input the unary encoding of the security parameter and produces a pair* $(\mathsf{sk}, \mathsf{pk}) \in \mathcal{SK} \times \mathcal{PK}$ *containing a secret and a public key.*
- $\mathsf{Eval}_{\mathsf{VF}}$*: The evaluation algorithm takes as input a tuple* (sk, x) *containing a secret key* $\mathsf{sk} \in \mathcal{SK}$ *and an evaluation point* $x \in \{0,1\}^\lambda$ *and produces an tuple* (y, π) *containing an image* $y \in \mathcal{Y}$ *and a corresponding proof* $\pi \in \mathcal{P}$*.*
- $\mathsf{Vfy}_{\mathsf{VF}}$*: The verification algorithm takes as input a tuple* (pk, x, y, π) *containing a public key* $\mathsf{pk} \in \mathcal{PK}$*, an evaluation point* $x \in \{0,1\}^\lambda$*, an image* $y \in \mathcal{Y}$ *and a proof* $\pi \in \mathcal{P}$ *and returns a bit* b *denoting acceptance or rejection.*

A VF can assume a range of different attributes. Based on the satisfied properties we arrive at specific constructions such as verifiable random functions (VRFs) or verifiable unpredictable functions (VUFs). We first describe the correctness property of a VF. Informally a VF is correct if for any key-pair produced honestly from the key-generation algorithm and any point $x \in \{0,1\}$ the output (y, π) of the evaluation alogrithm is accepted by the verification algorithm.

Definition 2.4 (ε-Correctness) *A verifiable function* $\mathsf{VF} = (\mathsf{Gen}_{\mathsf{VF}}, \mathsf{Eval}_{\mathsf{VF}}, \mathsf{Vfy}_{\mathsf{VF}})$ *is ε-correct, if for all security parameters $\lambda \in \mathbb{N}^+$ we have that*

$$\Pr\left[\mathsf{Vfy}_{\mathsf{VF}}(\mathsf{pk}, x, y, \pi) = 1 : \begin{array}{r}(\mathsf{sk}, \mathsf{pk}) \leftarrow_{\$} \mathsf{Gen}_{\mathsf{VF}}(1^\lambda) \\ x \leftarrow_{\$} \{0,1\}^\lambda \\ (y, \pi) \leftarrow_{\$} \mathsf{Eval}_{\mathsf{VF}}(\mathsf{sk}, x)\end{array}\right] \geq \varepsilon(\lambda)$$

An ε-correct VF is defined to be perfectly correct, or just correct, if $\varepsilon = 1$, and it is defined to be statistically correct if $\varepsilon = 1 - \mu$ for a negligible function μ.

While correctness requires all honestly generated images to be accepted by the verification algorithm we additionally demand that the verification algorithm rejects all maliciously generated images. We define the property of unique provability to be the property that requires that for every evaluation point $x \in \{0,1\}^\lambda$, and every public-key $\mathsf{pk} \in \mathcal{PK}$ only one image $y \in \mathcal{Y}$ is accepted by the verification algorithm.

Definition 2.5 (Unique Provability) *A verifiable function* VF*, with algorithms* $(\mathsf{Gen}_{\mathsf{VF}}, \mathsf{Eval}_{\mathsf{VF}}, \mathsf{Vfy}_{\mathsf{VF}})$*, defined to be uniquely provable if for all evaluation points* $x \in \{0,1\}^\lambda$*, a public-keys* $\mathsf{pk} \in \mathcal{PK}$ *(even maliciously crafted ones) and any two images-proof pairs* $(y_0, \pi_0), (y_1, \pi_1) \in \mathcal{Y} \times \mathcal{PK}$*:*

$$\mathsf{Vfy}_{\mathsf{VF}}(\mathsf{pk}, x, y_0, \pi_0) = 1 \land \mathsf{Vfy}_{\mathsf{VF}}(\mathsf{pk}, x, y_0, \pi_0) = 1 \implies y_0 = y_1$$

We define a secure VF to be a VF that satisfies both correctness and unique provability. We define the computational security properties of pseudorandomness, weak pseudorandomness and unpredictability.

maybe separate correct and secure here or maybe omit it fully

Definition 2.6 ($(\mathsf{t}, \varepsilon)$-Pseudorandomness) *A VF* $(\mathsf{Gen}_{\mathsf{VF}}, \mathsf{Eval}_{\mathsf{VF}}, \mathsf{Vfy}_{\mathsf{VF}})$ *is defined to be* $(\mathsf{t}, \varepsilon)$*-pseudorandom if for all security parameters* $\lambda \in \mathbb{N}$*, for all stateful adversaries* $\mathcal{A}$*, running in time at most* $\mathsf{t}(\lambda)$*, the advantage of* $\mathcal{A}$ *in the prf security experiment outlined in* **??** *is at most* $\varepsilon(\lambda)$*, i.e. if*

$$\mathbf{Adv}^{\mathrm{prf}}_{\mathsf{VF}}(\lambda, \mathcal{A}) := 2 \cdot \left| \Pr\left[\mathbf{Exp}^{\mathrm{prf}}_{\mathsf{VF}}(\lambda, \mathcal{A}) = 1\right] - \frac{1}{2} \right| \leq \varepsilon(\lambda)$$

We say that a VF is pseudorandom, if it is $(\mathsf{t}, \varepsilon)$*-pseudorandom with* ε *being a negligible function and* t *being a polynomial, i.e.* $\varepsilon \in \mathsf{negl}(\lambda), \mathsf{t} \in \mathsf{poly}(\lambda)$*.*

In contrast to pseudorandomness, which requires that images of chosen evaluation points are pseudorandom, i.e. indistinguishable from uniformly random elements in the co-domain, weak pseudorandomness requires only that the evaluation of uniformly random evaluation points.

Definition 2.7 ($(\mathsf{t}, \varepsilon)$-weakly Pseudorandomness) *A VF* $(\mathsf{Gen}_{\mathsf{VF}}, \mathsf{Eval}_{\mathsf{VF}}, \mathsf{Vfy}_{\mathsf{VF}})$ *is defined to be* $(\mathsf{t}, \varepsilon)$*-weakly pseudorandom if for all security parameters* $\lambda \in \mathbb{N}$*, for all stateful adversaries* $\mathcal{A}$*, running in time at most* $\mathsf{t}(\lambda)$*, the advantage of* $\mathcal{A}$ *in the wprf security experiment outlined in* **??** *is at most* $\varepsilon(\lambda)$*, i.e. if*

$$\mathbf{Adv}^{\mathrm{wprf}}_{\mathsf{VF}}(\lambda, \mathcal{A}) := 2 \cdot \left| \Pr\left[\mathbf{Exp}^{\mathrm{wprf}}_{\mathsf{VF}}(\lambda, \mathcal{A}) = 1\right] - \frac{1}{2} \right| \leq \varepsilon(\lambda)$$

We say that a VF is weakly pseudorandom, if it is $(\mathsf{t}, \varepsilon)$*-weakly pseudorandom with* ε *being a negligible function and* t *being a polynomial, i.e.* $\varepsilon \in \mathsf{negl}(\lambda), \mathsf{t} \in \mathsf{poly}(\lambda)$*.*

Experiment $\mathbf{Exp}^{\text{prf}}_{\mathsf{VF}}(\lambda, \mathcal{A})$

$b \leftarrow_{\$} \{0,1\}$
$(\mathsf{sk}, \mathsf{pk}) \leftarrow_{\$} \mathsf{Gen}_{\mathsf{VF}}(1^\lambda)$
$S \leftarrow \varnothing$
$(x^*, \sigma) \leftarrow_{\$} \mathcal{A}^{\mathcal{O}_{\text{eval}}}(1^\lambda, \mathsf{pk})$
$(y_0, \pi_0) \leftarrow \mathsf{Eval}_{\mathsf{VF}}(\mathsf{sk}, x^*)$
$y_1 \leftarrow_{\$} \mathcal{Y}$
$b' \leftarrow_{\$} \mathcal{A}(1^\lambda, x^*, \sigma, y_b)$
return $b = b' \wedge x^* \notin S$

Experiment $\mathbf{Exp}^{\text{wprf}}_{\mathsf{VF}}(\lambda, \mathcal{A})$

$b \leftarrow_{\$} \{0,1\}$
$(\mathsf{sk}, \mathsf{pk}) \leftarrow_{\$} \mathsf{Gen}_{\mathsf{VF}}(1^\lambda)$
$S \leftarrow \varnothing$
$\sigma \leftarrow_{\$} \mathcal{A}^{\mathcal{O}_{\text{eval}}}(1^\lambda, \mathsf{pk})$
$x^* \leftarrow_{\$} \{0,1\}^\lambda$
$(y_0, \pi_0) \leftarrow \mathsf{Eval}_{\mathsf{VF}}(\mathsf{sk}, x^*)$
$y_1 \leftarrow_{\$} \mathcal{Y}$
$b' \leftarrow_{\$} \mathcal{A}^{\mathcal{O}_{\text{eval}}}(1^\lambda, x^*, \sigma, y_b)$
return $b = b' \wedge x^* \notin S$

Experiment $\mathbf{Exp}^{\text{up}}_{\mathsf{VF}}(\lambda, \mathcal{A})$

$(\mathsf{sk}, \mathsf{pk}) \leftarrow_{\$} \mathsf{Gen}_{\mathsf{VF}}(1^\lambda)$
$S \leftarrow \varnothing$
$(x^*, y^*) \leftarrow_{\$} \mathcal{A}^{\mathcal{O}_{\text{eval}}}(1^\lambda, \mathsf{pk})$
$(y, \pi) \leftarrow \mathsf{Eval}_{\mathsf{VF}}(\mathsf{sk}, x^*)$
return $y^* = y \wedge x^* \notin S$

Oracle $\mathcal{O}_{\text{eval}}(x)$

$S \leftarrow S \cup \{x\}$
$(y, \pi) \leftarrow_{\$} \mathsf{Eval}_{\mathsf{VF}}(\mathsf{sk}, x)$
return y

Figure 2.1: Security game for pseudorandomness, weak pseudorandomness and unpredictability. The differences between the prf and wprf security experiment are highlighted in gray.

Definition 2.8 ((ε, t)-Unpredictability) *A VF* $(\mathsf{Gen}_{\mathsf{VF}}, \mathsf{Eval}_{\mathsf{VF}}, \mathsf{Vfy}_{\mathsf{VF}})$ *is defined to be* $(\mathfrak{t}, \varepsilon)$*-unpredictable if for all security parameters* $\lambda \in \mathbb{N}$*, for all stateful adversaries* $\mathcal{A}$*, running in time at most* $\mathfrak{t}(\lambda)$*, the advantage of* $\mathcal{A}$ *in the up security experiment outlined in* **??** *is at most* $\varepsilon(\lambda)$*, i.e. if*

$$\mathbf{Adv}^{\text{up}}_{\mathsf{VF}}(\lambda, \mathcal{A}) := 2 \cdot \left| \Pr\left[\mathbf{Exp}^{\text{up}}_{\mathsf{VF}}(\lambda, \mathcal{A}) = 1\right] - \frac{1}{2} \right| \leq \varepsilon(\lambda)$$

We say that a VF is unpredictable, if it is $(\mathfrak{t}, \varepsilon)$*-unpredictable with* ε *being a negligible function and* $\mathfrak{t}$ *being a polynomial, i.e.* $\varepsilon \in \mathsf{negl}(\lambda), \mathfrak{t} \in \mathsf{poly}(\lambda)$*.*

We define a verifiable random function (VRF) to be a secure pseudorandom VF, a verifiable unpredictable function (VUF) to be a secure unpredictable VF and a weaker verifiable random function (wVRF) to be a secure weakly pseudorandom VF. We formally define the primitives VRF, VUF and wVRF in the following definitions.

Definition 2.9 (Verifiable Random Functions) *A verifiable random function* $(\mathsf{Gen}_{\mathsf{VRF}}, \mathsf{Eval}_{\mathsf{VRF}}, \mathsf{Vfy}_{\mathsf{VRF}})$ *is a VF that satisfies perfect correctness, unique provability and pseudorandomness.*

Definition 2.10 (Verifiable Unpredictable Functions) *A verifiable random function* $(\mathsf{Gen}_{\mathsf{VRF}}, \mathsf{Eval}_{\mathsf{VRF}}, \mathsf{Vfy}_{\mathsf{VRF}})$ *is a verifiable function that satisfies perfect correctness, unique provability and unpredictability.*

Definition 2.11 (Weak Verifiable Random Functions) *A verifiable random function* $(\mathsf{Gen}_{\mathsf{VRF}}, \mathsf{Eval}_{\mathsf{VRF}}, \mathsf{Vfy}_{\mathsf{VRF}})$ *is a verifiable function that satisfies perfect correctness, unique provability and weak pseudorandomness.*

Unbiasability, following the definition of **[?]**, is a property for VRFs that prevents adversaries from skewing the output distribution of the evaluation function on malleable keys. To formally definine unbiasability we require the notion of monotone predicates.

Definition 2.12 (Monotone Predicates [?]) *A probabilistic predicate* ρ *on the domain* $(\mathcal{Y} \cup \{\bot\})^*$ *is monotone, iff for all* $y \in \mathcal{Y}^*$*, all index sets* $I \subseteq [|y|]$ *and every bitstring of randomness* $r \in \{0,1\}^\lambda$ *we have*

$$\Pr[\rho(\tilde{y}; r) = 1] \leq \Pr[\rho(y; r) = 1]$$

where $\tilde{y}_i = y_i$ *if* $i \notin I$ *and* $\tilde{y}_i = \bot$ *otherwise.*

Definition 2.13 (Unbiasability [?]) *A VRF* VRF *is (fully)* $(\mathfrak{t}, \varepsilon, \mathfrak{a}, \mathfrak{b})$*-unbiasable if for every stateful adversary* $\mathcal{A}$ *running in time at most* $\mathfrak{t}$ *its advantage in the unbiasability security experiment in* **??** *is bounded by*

$$\mathbf{Adv}^{\mathrm{fub}}_{\mathsf{VRF}}(\lambda, \mathcal{A}) := 2 \cdot \Pr\left[\mathbf{Exp}^{\mathrm{fub}}_{\mathsf{VRF}}(\lambda, \mathfrak{a}, \mathfrak{b}, \mathcal{A}) = 1\right] - 1 \leq \varepsilon(\lambda)$$

We say that a VRF is unbiasable if it is $(\mathfrak{t}, \varepsilon, \mathfrak{a}, \mathfrak{b})$*-unbiasable and* ε *is a negligible function in the security parameter* λ*, i.e.* $\varepsilon \in \mathsf{negl}(\lambda)$*.*

Key-homomorphism of VF requires that the secret-key space and the product space of the image and proof space admit group structures $(\mathcal{SK}, \odot)$ and $(\mathcal{Y} \times \mathcal{P}, \otimes)$ respectively. Intuitively a VF is key-homomorphic if for every evaluation point $x \in \{0,1\}^\lambda$ the evaluation algorithm on x implements a grouphomomorphism in the secret key.

Definition 2.14 (Key-Homomorphism) *A VF* VF *is key-homomorphic if for every evaluation point* $x \in \{0,1\}^\lambda$*, secret key* $\mathsf{sk}_1, \mathsf{sk}_2 \in \mathcal{SK}$ *we have*

$$\mathsf{Eval}_{\mathsf{VF}}(\mathsf{sk}_1 \odot \mathsf{sk}_2, x) = \mathsf{Eval}_{\mathsf{VF}}(\mathsf{sk}_1, x) \otimes \mathsf{Eval}_{\mathsf{VF}}(\mathsf{sk}_2, x)$$

Experiment $\mathbf{Exp}^{\mathsf{fub}}_{\mathsf{VF}}(\lambda, \mathcal{A})$

$b \leftarrow_{\$} \{0,1\}$
$(\rho, \mathsf{pk}_1, \ldots, \mathsf{pk}_{\mathfrak{a}(\lambda)}) \leftarrow_{\$} \mathcal{A}(1^\lambda)$
require ρ is monotone
$x_1, \ldots, x_{\mathfrak{b}(\lambda)} \leftarrow_{\$} \{0,1\}^\lambda$
$(y^0_{1,1}, \pi^0_{1,1}), \ldots, (y^0_{\mathfrak{a}(\lambda),\mathfrak{b}(\lambda)}, \pi^0_{\mathfrak{a}(\lambda),\mathfrak{b}(\lambda)}) \leftarrow_{\$} \mathcal{A}(x_1, \ldots, x_{\mathfrak{b}(\lambda)})$
for $i \in [\mathfrak{a}(\lambda)], j \in [\mathfrak{b}(\lambda)]$ **do**
  $\beta_{i,j} \leftarrow \mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}_i, x_j, y^0_{i,j}, \pi^0_{i,j})$
  if $\beta_{i,j} = 1$ **then**
    $z_{i,j} \leftarrow y^b_{i,j}$
  else $z_{i,j} \leftarrow \perp$
$r \leftarrow \{0,1\}^\lambda$
return $\rho(z_{1,1}, \ldots, z_{\mathfrak{a}(\lambda),\mathfrak{b}(\lambda)}, r) \neq b$

Figure 2.2: Full unbiasability security experiment.

We also refer to key-homomorphism as secret-key aggregability. To define public-key aggregability we first define the cryptoraphic primitive of an aggregable verifiable function (AggVF) that additionally implements algorithms to combine image-proof pairs into a single image-proof pair aggregate.

think I should abandon the subsetting of the primitive and introduce . notation. Also use subexp **P** for primitive in combiner

2.3 Robust Combiners

We write robust combiner for robust $(n;k)$ P-combiner whenever n, k and P are clear from the context. A robust combiner gets the candidate schemes as a description of their turing machine and can therefore be adaptive in the code of its candidates. Note that this definition allows the robust combiner to ignores its candidate schemes and implement a secure instance of the underlying primitive indepdently of its candidates. One can therefore always construct a robust combiner for primitives that admit secure instances, by contraposition this also means that any result that rules out the existence of a robust combiner for a specific primitive P also rules out the existence of the primitive itself. Intuitively we would like for the combiner to derive its security from the security of its secure candidate schemes, therefore we define robust black-box combiner. A robust black-box combiner is a robust combiner that, instead of getting access to the description of the Turing machines of the candidate schemes, only gets oracle access to the underlying candidate schemes. We define the notion of a robust black-box combiner closely following the definition by [**?**]. Note that we explicitly require a robust black-box combiner to derive its security from its candidates, for that reason we require the security to hold relative to a PSPACE oracle PSPACE.

Very unhappy with this definition

Definition 2.15 ($(n;k)$-Robust Black-Box P-Combiner) *Given a cryptographic primitive* P *consisting of p algorithm* $(\mathsf{alg_P}^1, \ldots, \mathsf{alg_P}^p)$*, a robust* $(n;k)$ *black-box* P*-combiner is a p-tuple of oracle Turing machines with access to n oracles, the candidate schemes* $\mathcal{O}_1, \ldots, \mathcal{O}_n$

$$\Pi_{\mathsf{P}}^{\mathcal{O}_1,\ldots,\mathcal{O}_n} = \left(\Pi_{\mathsf{P}}^{\mathcal{O}_1,\ldots,\mathcal{O}_n}.\mathsf{alg_P}^1, \ldots, \Pi_{\mathsf{P}}^{\mathcal{O}_1,\ldots,\mathcal{O}_n}.\mathsf{alg_P}^n\right)$$

A $(n;k)$*-robust combiner* $\Pi_{\mathbf{P}}$ *for primitive P is defined to be black-box, denoted by* Π_{P}*, if additionally it satisfies that if there exists an adversary* $\mathcal{A}$ *(possibly computationally unbounded) that breaks the security of* $\Pi_{\mathsf{P}}^{\mathcal{O}_1,\ldots,\mathcal{O}_n}$*, then there exist adversaries* $\mathcal{B}_1, \ldots, \mathcal{B}_{n+k-1}$ *with oracle access to* $\mathcal{A}$ *for at least* $n-k+1$ *different candidates.*

We remark that compositite security definitions that rely on multiple properties are broken if there exists an adversary that breaks a security property with significant advantage. Note that if the security would not hold relative to a PSPACE oracle PSPACE then we would find that a secure implementation of the primitive would indeed implement a secure robust black-box combiner since there exists no adversary breaking the security properties of P.

Chapter 3

Results

3.1 Robust Black-Box DAG Combiner

Intuitively, a robust black-box combiner operates on n candidate schemes by interleaving oracle calls to the candidates implemented functions with processing to combine their outputs in a way such that security properties of the secure subset of oracles are preserved if there are sufficiently many of them. To argue about the impossibility of such constructions it is helpful to impose additional structure on the combiner, to structure it as a directed acyclic graph modeling the computation of the combiner. In the DAG every oracle query corresponds to a vertex colored with the appropriate label Viewing a robust black-box combiner as a robust black-box DAG combiner is therefore simply a way make the structured execution of a combiner explicit, making the reasoning about oracle query dependencies and oracle-independent computation explicit. Additionally since robust black-box combiners need to satisfy the security properties relative to a PSPACE oracle, in the later proof we rely on the ability of the PSPACE oracle to brute-force many secret keys in the algorithm if there are not queries to oracles. For this line of reasoning this description is also more convenient. As we will show later in **??**, the notions of robust black-box combiners and robust black-box DAG combiners are equivalent.

We define robust black-box DAG combiners by representing each implemented algorithm alg with a directed acyclic graph $\mathcal{G}_{\mathsf{alg}} = (V_{alg}, E_{alg})$ with a source $s_{\mathsf{alg}} \in V_{\mathsf{alg}}$ that has no in-going edges, corresponding to the input of the algorithm, and a sink $t_{alg} \in V_{alg}$ with no out-going edges, corresponding to a final processing step. Additionally the description of the DAG combiner contains a vertex labling $l_{alg} : V_{alg} \setminus \{s_{alg}, t_{alg}\} \to [n] \times \{alg_1, \ldots, alg_k\}$ which maps every vertex that is not a source or a sink to the corresponding candidate oracle and the specific function called on an input $\mathbf{x}_v$ to produce an output $\mathbf{y}_v$. We define the output $\mathbf{y}_{s_{alg}}$ of the source to be the combiner

Algorithm $[\![\mathcal{G}, c, s, t, \{f_v\}_{v\in V(\mathcal{G})}; \rho]\!](\mathbf{x})$

1 : $\mathbf{y}_s \leftarrow \mathbf{x}$
2 : **for** $v \in \tau(\mathcal{G}, s) \setminus \{s, t\}$ **do**
3 : $\quad \mathbf{x}_v \leftarrow f_v(\mathbf{x}, \{(\mathbf{y}_w, w)\}_{w\in\mathcal{N}^-(v)}; \rho)$
4 : $\quad \mathbf{y}_v \leftarrow \mathcal{O}_{c(v)}(\mathbf{x}_v)$
5 : **return** $f_t(\mathbf{x}, \{\mathbf{y}_w\}_{w\in\mathcal{N}^-(t)})$

Algorithm $\tau(\mathcal{G}, s)$

1 : // Topological ordering TODO

Figure 3.1: Interpretation of $\left(\mathcal{G}, c, s, t, \{f_v\}_{v\in V(\mathcal{G})}\right)$ into an algorithm. τ returns the vertices in $\mathcal{G}$ in topological order starting from the source s.

algorithm input and the output of the sink t_{alg} to be the output of the combiner. The input $\mathbf{x}_v$ to the query at vertex $v \in V_{alg} \setminus \{s_{alg}, t_{alg}\}$ is computed by an efficient (ppt.) Turing machine f^v_{alg} associated with vertex v in G_{alg} which takes as input the labeled outputs of all predecessors $w \in \mathcal{N}_{in}(v)$ and produces an oracle query $\mathbf{x}_v$. We formally describe robust black-box DAG combiners in the following definition.

Definition 3.1 (Robust $(n;k)$ Black-Box DAG P-Combiner) *Given a cryptographic primitive* P *implementing a set of algorithms indexed by $j \in [|\mathsf{P}|]$. A robust $(n;k)$ black-box DAG* P*-combiner is a $|\mathsf{P}|$-tuple*

$$\left(\mathfrak{G}_j = \left(\mathcal{G}_j, c_j, s_j, t_j, \{f_{j,v}\}_{v\in V(\mathcal{G}_j)}\right)\right)_{j\in[|\mathsf{P}|]}$$

where $\mathcal{G}_j$ is a directed acyclic graph with vertex set $V(\mathcal{G}_j)$, $c_j : V(\mathcal{G}_j) \setminus \{s_j, t_s\} \to [n] \times [|\mathsf{P}|]$ is a vertex-coloring with colors (a, b) indicating that the corresponding query is made to algorithm a of candidate oracle b, source and sink $s_j, t_s \in V(\mathcal{G}_j)$ satisfying $\mathcal{N}^-(s_j) = \mathcal{N}^+(t_j) = \varnothing$ and for simplicity the only vertices satisfying those properties and finally $\{f_{j,v}\}_{v\in V(\mathcal{G}_j)}$ are ppt. Turing machines. satisfying that the following tuple of oracle Turing machines is a robust $(n;k)$ black-box combiner.

$$\widetilde{\Pi^{\mathcal{O}_1,\ldots,\mathcal{O}_n}_{\mathsf{P}}} = \left(\widetilde{\Pi^{\mathcal{O}_1,\ldots,\mathcal{O}_n}_{\mathsf{P}}}.[\![\mathfrak{G}_1]\!], \ldots, \widetilde{\Pi^{\mathcal{O}_1,\ldots,\mathcal{O}_n}_{\mathsf{P}}}.[\![\mathfrak{G}_{|\mathsf{P}|}]\!]\right)$$

Where $[\![\mathfrak{G}_j]\!]$ is the interpretation according to **??**.

A robust black-box DAG combiner algorithm is then executed by tracing the computation through the corresponding DAG G_{alg}, starting at the source s_{alg} and ending at the sink t_{alg} in a topological ordering. The interpretation of an algorithm alg described by the robust black-box DAG combiner by $(\mathcal{G}_{alg}, s_{alg}, t_{alg}, l_{alg}, \{f^v_{alg}\}_{v\in V_{alg}\setminus\{s_{alg},t_{alg}\}})$ is shown in **??**.

Example. We introduce the notion of robust black-box DAG combiners to make the underlying structure of robust black-box combiners more explicit. To illustrate the definition we will show the structure of the DAG combiner of a robust $(2;1)$ black-box combiner **??** in **??**. We label every vertices v in the DAGs that is not a source or sink with $l_{alg}(v)$, i.e. their vertex labling and for every vertex add the functional description of the Turing machines f^{v}_{alg}.

$$s_{gen}$$

$$(0, gen), \mathbf{y}_{s_{gen}} \mapsto \quad (1, gen)$$

$$t_{gen}$$

Figure 3.2: Drawing of the graphs $\mathcal{G}$

Since later on we will focus on showing the impossibility of certain combiner constructions it's essential that the notion of combiner that we use is as strong as possible. By definition a robust black-box DAG combiner is also a robust black-box combiner. We show that the notions of robust black-box DAG combiner and robust black-box combiner are equally strong and capture the same set of constructions. We formulate this result in the following theorem.

Theorem 3.2 (Generality of Robust $(n;k)$ Black-Box DAG Combiners) *For every robust $(n;k)$ black-box combiner there exists a computationally equivalent robust $(n;k)$ black-box DAG combiner.*

Intuition. We need to show that for every robust $(n;k)$ black-box combiner Π we can construct a robuts $(n;k)$ black-box DAG combiner $\widetilde{\Pi}$ that represents the same function if interpreted, given that Π is efficient is also efficient. The proof works by unrolling the computation done by the combiner Π for every oracle call made to a candidate scheme. Since we know that Π is a black-box combiner, it runs in time polynomial in λ, n, m. Let $T \in \mathsf{poly}(\lambda, n, m)$ be an upper bound on the number of steps that the algorithm runs over all inputs $\mathbf{x}$ into the combiner. We start by constructing a graph $\mathcal{G}_{alg}$ that consists of T layers of nm vertices, where n is the number of candidates and m is the number of algorithms implemented by the primitive. On every layer, each vertex corresponds to an oracle call to a specific oracle algorithm of a candidate. We connect two vertices $v, w \in V_{alg}$ if the they are not in the same layer with an edge pointing from the vertex in the layer closer to the source to the other vertex. To define the functions f^v_{alg} for $v \in V_{alg}$, a vertex in layer $k \in [T]$, we construct a Turing machine that simulates the robust black-box combiner Π until the k'th oracle query. Once the $i \in [k-1]$'th oracle query is made we instead use the correct output of the correct vertex in the i'th layer and move on. For the k'th query we simply query the oracle of our vertex if the oracle is correct (and also if its is not correct, this will then simply be ignored later one during the execution).

To prove that this construction results in the same output as the robust black-box combiner does, we proceed by induction on the layer depth of the current query, claiming that the output of the correct vertex at that layer aligns with the query output of the corresponding query in the execution of the robust black-box combiner. Note that the DAG only contains $T \times n \times m$ many vertices and Turing machines f^v_{alg}. Since the computation at every vertex is bounded from above by the time it takes the robust black-box combiner to run and we only make one oracle call at every vertex, the result of every vertex can be computed in time at most T. Overall the time it takes for the robust black-box DAG combiner to run is therefore polynomial in λ, n, m. We provide a formal proof below.

Proof (??) Let P be a cryptographic primitive Π_{P} with defining algorithms $(p_1, \ldots, p_{|\mathsf{P}|})$, et Π_{P} be a robust $(n;k)$ black-box P-combiner according to **??**, and let $j \in [|\mathsf{P}|]$ be arbitrary. By **??** there exists a polynomial $T \in \mathsf{poly}(n, \lambda, |x|)$ where x is the input to p_j. We construct a directed acyclic graph $\mathcal{G}_j$

$$\mathcal{G}_j = ([n] \times [|\mathsf{P}|] \times [T] \cup \{s, t\}, \{\, ((a_1, b_1, t_1), (a_2, b_2, t_2)) : t_1 < t_2 \,\})$$

We define the coloring by $c(a, b, t) = (a, b)$. We define the Turing machines $\{f_{j,v}\}_{v \in V(\mathcal{G}_j)}$ iteratively starting from the beginning of the computation s, we define $f_{j,v_{j,a,b,1}}$ to be the ppt Turing machine that simulates the $\Pi_{\mathsf{P}}.p_j$ until the first call to its oracle. If that oracle happens to be (a, b) then we end, otherwise write 0 onto the tape. In every subsequent level $f_{j,a,b,t+1}$ simulates the combiner on and answers the queries to the respective oracle with the output of that query ($y_{j,a,b,k}$ for $k \in [t]$) by progressively simulating all previous Turing machines $f_{j,a,b,k}$ for $k \in [t]$. By checking whether after the computation we find a 0 on the tape we can trace the path of queried oracles. We can now continue the simulation of the combiner since we know exactly the state of the tape. The combiner is necessarily efficient since for every the number of vertices in the graph is bounded from above by $nT(n, \lambda, |x|)|\mathsf{P}|$ where $T \in \mathsf{poly}(m, \lambda, |x|)$, we only make exactly one query to any oracle per vertex (which also puts a polynomial bound on the number of oracle calls), the topological ordering can be computed in polynomial time. We are only left to show that the Turing machines $\{f_{j,v}\}_{v \in V(\mathcal{G}_j)}$ are indeed ppt. To this end note that $f_{j,v}$ simulates the black-box combiner Turing machine, which is assumed to be ppt, at most once where it can answer any oracle query by looking up the corresponding output of the layer in time polynomial in $n, |\mathsf{P}|$ and λ. □

3.2 Impossibility of Robust Black-Box VRF-Combiner

We now tackle the main challenge of combining VRFs in a black-box fashion. Specifically we study the possibility of constructing a robust $(n;k)$ black-box combiner Π_{VRF} for the VRF primitive. During this section we will make the additional assumption that the secret key space of all candidate schemes is equivalent (or at least is in bijective correspondence to) $\{0,1\}^{s(\lambda)}$ for s a polynomial in λ.

more introduction

Warm Up. As a warm-up we will start by showing the impossibility of a robust $(2;1)$ black-box combiner for VRFs. That is, we show that there exists no efficient (ppt.) Turing machine Π_{VRF} such that for any two oracles $\mathcal{O}_1, \mathcal{O}_2$ implementing a VF, $\Pi_{\mathsf{VRF}}^{\mathcal{O}_1,\mathcal{O}_2}$ implements a secure VRF if at least one of the candidates implements a secure VRF. We state the following lemma.

Lemma 3.3 (Robust (2;1) Black-Box VRF Combiner) *There exists no robust* $(2;1)$ *black-box combiner in the sense of* **??** *for the VRF primitive defined in* **??**.

Intuition. We must show that for every construction Π_{VRF} implemented by a ppt. Turing machine there exist two oracles $\mathcal{O}_1, \mathcal{O}_2$, of which at least one implements a secure VRF, such that Π_{VRF} is either not pseudorandom in the sense of **??**, perfectly correct in the sense of **??** or not uniquely provable in the sense of **??**. Applying the result from **??** we can argue about the non-existence of DAG combiners instead, fix an arbitrary DAG combiner Π_{VRF} and consider the DAG corresponding to the evaluation algorithm $\mathcal{G}_{eval}$. We assume that the combiner is uniquely provable and perfectly correct and construct an adversary $\mathcal{A}$ that wins the prf security experiment against the combiner with significant advantage. For that, the adversary will employ the PSPACE oracle PSPACE to compute a set of candidate secret keys by iteratively filtering the secret key space on a polynomial number of evaluation points the adversary obtains in the first stage of the prf experiment. To execute the filtering the adversary constructs a ppt. Turing machine *eval* that takes as input a tuple (sk^*, x), the combiner input, and returns (y, π) such that the output aligns with the combiner output if sk^* is indeed the secret key input and with high probability if the sk^* is a different secret key. Importantly *eval* must not make any queries to the candidate oracles, that way we can use the PSPACE complete oracle and an a tuple (x, y, π) to compute:

$$\mathcal{SK}' = \left\{\mathsf{sk} \in \mathcal{SK} : eval^{\mathsf{PSPACE}}(\mathsf{sk}, x) = (y, \pi)\right\}$$

Assuming that we can efficiently compute *eval* we can refine the set of candidates $2s(\lambda)$ times. Now assume that we eliminate at least half the candidates in every second iteration, then we end up with a polynomial

set of candidates, in which the true secret key is contained. The adversary can now guess the correct secret key with non-negligible probability and win the pseudorandomness experiment with non-negligible advantage. If on the other hand the set of candidates doesnt shrink at half in at least every second step then intuitively on random inputs x a lot of secret keys (whether or not they are the correct secret key) result in the same output of *eval*, which with non-negligible probability agrees with the combiner. We can still simply sample a secret key of this set of candidates to obtain a secret key sk, with possible negligible chance to be the correct secret key, but non-negligible probability to result in the same output y on a random x. This again enables the adversary to beat the prf security experiment with non-negligible advantage.

We now show how to construct the Turing machine *eval*, for that we first fix arbitrary oracles $\mathcal{O}_1, \mathcal{O}_2$ which are uniquely provable and perfectly correct, a secret key sk in the support of the key generation function and any evaluation point x. We now consider the vertices $v \in V_{eval}$ in a fixed topological ordering and classify them for *relevance* in the sense that a vertex is *relevant* if there exist two outputs $\hat{\mathbf{y}}_v \neq \mathbf{y}_v$ in the set of possible outputs of the corresponding oracle query (e.g. $\mathcal{Y}_{\mathcal{O}_1} \times \mathcal{P}_{\mathcal{O}_1}$ if the query is to evaluation of candidate $\mathcal{O}_1$) such that if we simulate the the execution of the combiners evaluation algorithm but replace the reply of the oracle with $\hat{\mathbf{y}}_v$ then we obtain a different output $\hat{\mathbf{y}}_{t_{eval}} \neq \mathbf{y}_{t_{eval}}$. We will for now assume that that every oracle query is only queried once on every distinct input $\mathbf{x}$, we'll show that this is the case later. The same classification of vertices we do for the verification algorithm.

Now initialize the second candidate with $\mathcal{O}_0$ and the first candidate with any secure VRF. To replace the oracle queries in the combiner and arrive at *eval* on a high level we argue that a vertex is relevant in the evaluation algorithm if and only if there exists a relevant vertex in the verification algorithm such that the queries made at that vertex is exactly the same as the one done in eval. Since every relevant query is repeated in the verification algorithm and every not repeated query in the verification is irrelevant we can simply define *eval* to be the Turing machine for (x, y, π, τ) where τ is the verification algorithms transcript on x, y, π that simulates the combiners evaluation algorith, if he encounters an oracle query to $\mathcal{O}_1$ the simulator checks the transcript τ if it contains the answer and answers with the answer if it exists. If it doesnt exist this means that either 1. the secret key doesn't correspond to (x, y, π) or and the query is possibly even relevant or 2. the secret key is correct, or related, and the query is irrelevant. If it encounters a query to $\mathcal{O}_2$ it the answer is known.

If we are in case 2., then we can simply iterate over the vertices in topological ordering and try every value if the query is not in the transcript until every combination of possibile outputs is tested, obtaining the same output

every time. Assume that were not the case then we could again break unique provability by making $\mathcal{O}_2$ secure again (vrf-ification) and break unique provability. If we are in case 1. then and for every guessing through the combiner (except for the recordings in τ) the output is y, then it is also y for the honest evaluation. We conclude that by only allowing those secret keys to survive at every step, we conserve *exactly* all secret keys which evaluate x to y, π.

Now we quickly need to argue that oracle queries are *unique* in the sense that a given algorithm of a given candidate is queried at most once on a specific input $\mathbf{x}$ and that we can construct a secure VRF from a uniquely provable and perfectly correct VRF that agrees on the original VRF on all secret keys in a polynomial set. For uniqueness the case is clear since the oracles need to be deterministic, assume they were not deterministic and that two outputs in the support would result in a different output, than we would arrive at a contradiction since by perfect correctness both must verify since they are honestly generated outputs and by unique provability not both must verify since they are distinct. We can therefore assume that the candidates are deterministic or if they are random we can just replace the random output with an arbitrary but fixed output in the support. The DAG combiner can now be slightly modified to store the all queries made, if a query is made a second time then instead of requerying he could just simply use the result from the first query (since in our DAG construction the Turing machine has access to *all* previous outputs).

For the VRF-ification, we give some intuition why a secure VRF can be remapped on a polynomial set of secret keys to assume the same output as a uniquely provable and perfectly correct VRF. For that remapping we introduce a symmetric and reflexive relation $\sim \subseteq \mathcal{SK} \times \mathcal{SK}$ defined as follows:

$$\mathsf{sk}_1 \sim \mathsf{sk}_2 \iff \exists \mathsf{pk} \in \mathcal{PK}, \Pr\begin{bmatrix}(\mathsf{sk}_1, \mathsf{pk}) \leftarrow_{\$} \mathsf{Gen}_{\mathsf{VRF}}(1^\lambda) \wedge \\ (\mathsf{sk}_2, \mathsf{pk}) \leftarrow_{\$} \mathsf{Gen}_{\mathsf{VRF}}(1^\lambda)\end{bmatrix} > 0$$

Let $\sim^* := \bigcup_{k \in \mathbb{N}} \sim^k$ be the transitive closure of $\sim$, since $\sim$ is reflexive and symmetric already $\sim^*$ is an equivalence relation. We claim that if two keys $\mathsf{sk}_1, \mathsf{sk}_2$ are in the same equivalence relation, i.e. $\mathsf{sk}_1 \sim^* \mathsf{sk}_2$ then $\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}_1, \cdot) = \mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}_1, \cdot)$ since otherwise we could break unique provability. If we now want to remap the outputs of a VRF $\mathcal{O}_{\mathbf{VRF}}$ on the set $\widetilde{\mathcal{SK}}$ then we need to remap the outputs of evaluation in $\widehat{\mathcal{SK}} := \bigcup_{\widetilde{\mathsf{sk}} \in \widetilde{\mathcal{SK}}} [\widetilde{\mathsf{sk}}]_{\sim^*}$, the union of all equivalence classes of $\widetilde{\mathcal{SK}}$. Since the newly mapped to points come from a uniquely provable and perfectly correct VF and the secure VRF is also a uniquely provable and perfectly correct VRF the resulting VF is uniquely provable and perfectly correct. To show that its pseudorandom we show that in the execution of the pseudorandomness game the probability

of sampling a secret key $\widehat{\mathsf{sk}} \in \widehat{\mathcal{SK}}$ is negligible if $|\widehat{\mathcal{SK}}| \in \mathsf{poly}(\lambda)$. Assume towards a contradiction that the following holds:

$$\Pr\left[\mathsf{sk} \in \widehat{\mathcal{SK}} : (\mathsf{sk}, \mathsf{pk}) \leftarrow_{\$} \mathsf{Gen}_{\mathsf{VRF}}(1^{\lambda})\right] > \frac{1}{p(\lambda)}$$

For a polynomial $p \in \mathsf{poly}(\lambda)$, then we could simply resample a secret key sk' which would also with probability at least $1/p(\lambda)$ be in $\widehat{\mathcal{SK}}$. Additionally with probability at least $1/|\widehat{\mathcal{SK}}| \in \mathsf{poly}(\lambda)$ we have $\mathsf{sk} \sim^{*} \mathsf{sk}'$ which means that with probability $1/p(\lambda)|\widehat{\mathcal{SK}}| \in \mathsf{poly}(\lambda)$ we obtain a secret key which allows us to recompute all outputs, breaking pseudorandomness.

Proof of ??. Let Π_{VRF} be an arbitrary robust $(2;1)$ black-box DAG combiner for the VRF primitive. We claim that there exists a related robust black-box DAG combiner for the VRF primitive that only queries every implementation function of every candidate scheme on any input $\mathbf{x}$ at most once.

Claim 3.4 *Let Π_{VRF} be a robust $(2;1)$ black-box DAG combiner for the VRF primitive, there exists a robust $(2;1)$ black-box DAG combiner for the VRF primitive $\widetilde{\Pi_{\mathsf{VRF}}}$ such that in its evaluation and verification algorithm DAG $\mathcal{G}_{eval}$ and $\mathcal{G}_{vfy}$ and each of the vertices $v \in V_{eval}$ and $w \in V_{vfy}$ the call to*

TODO: easy

Additionally we show that given two VF oracles $\mathcal{O}_{\mathbf{VRF}}$ and $\mathcal{O}_{up}$ such that $\mathcal{O}_{\mathbf{VRF}}$ implements a secure VRF and $\mathcal{O}_{up}$ implements a perfectly correct and uniquely provable VF we can construct a new oracle $\mathcal{O}_{\mathbf{VRF}}(\mathcal{O}_{up}, \widetilde{\mathcal{SK}})$ such that $\mathcal{O}_{\mathbf{VRF}}(\mathcal{O}_{up}, \widetilde{\mathcal{SK}}, \widetilde{\mathcal{PK}})$ implements a secure VRF and its evaluation algorithm agrees on all points (sk, x) with the oracle $\mathcal{O}_{up}$ where $\mathsf{sk} \in \widetilde{\mathcal{SK}}$ and $\mathsf{pk} \in \widetilde{\mathcal{PK}}$ if $|\widetilde{\mathcal{SK}}|, |\widetilde{\mathcal{PK}}| \in \mathsf{poly}(\lambda)$. Formally:

Claim 3.5 *Let $\mathcal{O}_{\mathbf{VRF}}$ be a VF oracle that implements a secure VRF in the sense of* **??**, *$\mathcal{O}_{up}$ be a VF oracle that is uniquely provable in the sense of* **??** *and perfectly correct in the sense of* **??** *such that $\mathcal{O}_{\mathbf{VRF}}$ and $\mathcal{O}_{up}$ are on the same spaces and let $\widetilde{\mathcal{SK}}, \widetilde{\mathcal{PK}}$ be polynomially bounded subsets of the secret and public key spaces of both oracles. There exists an oracle $\mathcal{O}_{mix}$ such that the following properties are satisfied:*

1. *The evaluation algorithm of $\mathcal{O}_{mix}$ agrees with the evaluation algorithm of $\mathcal{O}_{up}$ on all secret keys $\widetilde{\mathsf{sk}} \in \widetilde{\mathcal{SK}}$, i.e. we have that*

$$\forall \widetilde{\mathsf{sk}} \in \widetilde{\mathcal{SK}}, \mathcal{O}_{mix}.\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}, \cdot) = \mathcal{O}_{up}.\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}, \cdot)$$

2. *The verification algorithm of $\mathcal{O}_{mix}$ agrees with the evaluation algorithm of $\mathcal{O}_{up}$ on all public keys $\widetilde{\mathsf{pk}} \in \widetilde{\mathcal{PK}}$, i.e. we have that*

$$\forall \widetilde{\mathsf{pk}} \in \widetilde{\mathcal{PK}}, \mathcal{O}_{mix}.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}, \cdot, \cdot, \cdot) = \mathcal{O}_{up}.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}, \cdot, \cdot, \cdot)$$

3. *The oracle $\mathcal{O}_{mix}$ implements a secure VRF in the sense of* **??**.

Note first that it is not sufficient to simply change all of the mappings of the evaluation algorithm of $\mathcal{O}_{\mathbf{VRF}}$ on the points in $\widetilde{\mathcal{SK}}$ to the output of $\mathcal{O}_{up}$ and the mappings of the verification algorithm of $\mathcal{O}_{\mathbf{VRF}}$ on the points in $\widetilde{\mathcal{PK}}$ to the output of $\mathcal{O}_{up}$. Such a construction would indeed not necessarily be uniquely provable or perfectly correct since for one we might have not guaranteed that the verification of a newly mapped output succeeds or the verification of a previously mapped output rejects and also we need to consider that if two secret keys have non-zero probability to be sampled with the same public key, then they both need to induce the same mapping $\mathcal{X} \to \mathcal{Y} \times \mathcal{P}$, also this holds for the public key perspective if we change the mappings of the verification algorithm.

Proof of ??. Let $\sim \subseteq \mathcal{SK}^2$ be a relation defined as follows.

$$\mathsf{sk}_1 \sim \mathsf{sk}_2 \iff \exists \mathsf{pk} \in \mathcal{PK}, \Pr\begin{bmatrix}(\mathsf{sk}_1, \mathsf{pk}) \leftarrow_{\$} \mathsf{Gen}_{\mathsf{VRF}}(1^\lambda) \wedge \\ (\mathsf{sk}_2, \mathsf{pk}) \leftarrow_{\$} \mathsf{Gen}_{\mathsf{VRF}}(1^\lambda)\end{bmatrix} > 0$$

And consider $\sim^* = \bigcup_{k \in \mathbb{N}} \sim^k$, the transitive closure of $\sim$. We claim that $\sim^*$ is an equivalence relation as its the transitive closure of a reflexive and

symmetric relation $\sim$. We use $[\mathsf{sk}]_{\sim^*}$ for a secret key $\mathsf{sk} \in \mathcal{SK}$ to denote the equivalence class of sk under $\sim^*$, i.e. $\{\mathsf{sk}' \in \mathcal{SK} : \mathsf{sk} \sim^* \mathsf{sk}'\}$. Additionally for a public key $\mathsf{pk} \in \mathcal{PK}$ define $R_{\mathcal{PK}}(\mathsf{pk})$, the subset of secret keys *related* to pk and symmetrically for a secret key $\mathsf{sk} \in \mathcal{SK}$ define $R_{\mathcal{SK}}(\mathsf{sk})$, the subset of public keys *related* to sk.

$$R_{\mathcal{PK}}(\mathsf{pk}) := \left\{\mathsf{sk} \in \mathcal{SK} : \Pr\left[(\mathsf{sk}, \mathsf{pk}) \leftarrow_{\$} \mathsf{Gen}_{\mathsf{VRF}}(1^\lambda)\right] > 0\right\} \tag{3.1}$$

$$R_{\mathcal{SK}}(\mathsf{sk}) := \left\{\mathsf{pk} \in \mathcal{PK} : \Pr\left[(\mathsf{sk}, \mathsf{pk}) \leftarrow_{\$} \mathsf{Gen}_{\mathsf{VRF}}(1^\lambda)\right] > 0\right\} \tag{3.2}$$

We now define the oracle $\mathcal{O}_{mix}$ using the following implementation functions for its corresponding key generation function $\mathcal{O}_{mix}.\mathsf{Gen}_{\mathsf{VRF}}(1^\lambda)$, its evaluation function $\mathcal{O}_{mix}.\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}, x)$ and its verification function $\mathcal{O}_{mix}.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}, x, y, \pi)$ for that we first define the set of secret and public keys $\mathcal{SK}_{map}$ and $\mathcal{PK}_{map}$ respectively on which we need to remap $\mathcal{O}_{\mathbf{VRF}}$.

$$\mathcal{SK}_{map} := \bigcup_{\widetilde{\mathsf{sk}} \in \widetilde{\mathcal{SK}}} [\widetilde{\mathsf{sk}}]_{\sim^*_{\mathcal{SK}}} \cup \bigcup_{\widehat{\mathsf{pk}} \in \bigcup_{\widetilde{\mathsf{pk}} \in \widetilde{\mathcal{PK}}} [\widetilde{\mathsf{pk}}]_{\sim^*_{\mathcal{PK}}}} R_{\mathcal{PK}}(\widehat{\mathsf{pk}}) \tag{3.3}$$

$$\mathcal{PK}_{map} := \bigcup_{\widetilde{\mathsf{pk}} \in \widetilde{\mathcal{PK}}} [\widetilde{\mathsf{pk}}]_{\sim^*_{\mathcal{PK}}} \cup \bigcup_{\widehat{\mathsf{sk}} \in \bigcup_{\widetilde{\mathsf{sk}} \in \widetilde{\mathcal{SK}}} [\widetilde{\mathsf{sk}}]_{\sim^*_{\mathcal{SK}}}} R_{\mathcal{SK}}(\widehat{\mathsf{sk}}) \tag{3.4}$$

Now we define $\mathcal{O}_{mix}$ generation to simply be the VRF oracle key generation, we define the evaluation of $\mathcal{O}_{mix}$ to interpolate between the evaluation of $\mathcal{O}_{\mathbf{VRF}}$ and $\mathcal{O}_{up}$ based on whether the secret key is in $\mathcal{SK}_{mix}$ and finally we define the verification of $\mathcal{O}_{mix}$ to interpolate between the verification functions of $\mathcal{O}_{\mathbf{VRF}}$ and $\mathcal{O}_{up}$ based on whether the public key is in $\mathcal{PK}_{mix}$. We show the oracle in **??**.

$$\mathcal{O}_{mix}.\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}, x) = \begin{cases} \mathcal{O}_{up}.\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}, x) & \text{if } \mathsf{sk} \in \mathcal{SK}_{map} \\ \mathcal{O}_{\mathbf{VRF}}.\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}, x) & \text{otherwise} \end{cases} \tag{3.5}$$

$$\mathcal{O}_{mix}.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}, x, y, \pi) = \begin{cases} \mathcal{O}_{up}.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}, x, y, \pi) & \text{if } \mathsf{pk} \in \mathcal{PK}_{map} \\ \mathcal{O}_{\mathbf{VRF}}.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}, x, y, \pi) & \text{otherwise} \end{cases} \tag{3.6}$$

We first show that $\mathcal{O}_{mix}$ is uniquely provable and perfectly correct. Let sk, x be fixed and $(y, \pi) \leftarrow \mathcal{O}_{mix}.\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}, x)$, then if $\mathsf{sk} \in \mathcal{SK}_{map}$ we have $(y, \pi) \leftarrow \mathcal{O}_{up}.\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}, x)$ then for every $\mathsf{pk} \in \mathcal{PK}$ that can be sampled with sk we have $\mathsf{pk} \in R_{\mathcal{SK}}(\mathsf{sk})$, hence $\mathcal{O}_{mix}.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}, x, y, \pi) = \mathcal{O}_{up}.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}, x, y, \pi) = 1$ by perfect correctness of $\mathcal{O}_{up}$. If $\mathsf{sk} \notin \mathcal{SK}_{map}$ then $(y, \pi) \leftarrow \mathcal{O}_{\mathbf{VRF}}.\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}, x)$ and then for every pk that can be sampled $\mathsf{pk} \notin \mathcal{PK}_{map}$ since otherwise we would have

$$\mathsf{pk} \in \bigcup_{\widetilde{\mathsf{pk}} \in \widetilde{\mathcal{PK}}} [\widetilde{\mathsf{pk}}]_{\sim^*_{\mathcal{PK}}} \vee \mathsf{pk} \in \bigcup_{\widehat{\mathsf{sk}} \in \bigcup_{\widetilde{\mathsf{sk}} \in \widetilde{\mathcal{SK}}} [\widetilde{\mathsf{sk}}]_{\sim^*_{\mathcal{SK}}}} R_{\mathcal{SK}}(\widehat{\mathsf{sk}})$$

In case that $\mathsf{pk} \in \bigcup_{\widetilde{\mathsf{pk}} \in \widetilde{\mathcal{PK}}} [\widetilde{\mathsf{pk}}]_{\sim^*_{\mathcal{PK}}}$ we would have $\mathsf{sk} \in \bigcup_{\widehat{\mathsf{pk}} \in \bigcup_{\widetilde{\mathsf{pk}} \in \widetilde{\mathcal{PK}}} [\widetilde{\mathsf{pk}}]_{\sim^*_{\mathcal{PK}}}} R_{\mathcal{PK}}(\widehat{\mathsf{pk}})$ and therefore $\mathsf{sk} \in \mathcal{SK}_{map}$. If $\mathsf{pk} \in \bigcup_{\widehat{\mathsf{sk}} \in \bigcup_{\widetilde{\mathsf{sk}} \in \widetilde{\mathcal{SK}}} [\widetilde{\mathsf{sk}}]_{\sim^*_{\mathcal{SK}}}} R_{\mathcal{SK}}(\widehat{\mathsf{sk}})$ then $\mathsf{pk} \in R_{\mathcal{SK}}(\widehat{\mathsf{sk}})$ for at least some $\widehat{\mathsf{sk}} \in \bigcup_{\widetilde{\mathsf{sk}} \in \widetilde{\mathcal{SK}}} [\widetilde{\mathsf{sk}}]_{\sim^*_{\mathcal{SK}}}$, i.e. for some $\widehat{\mathsf{sk}}$ in an equivalence class of an element in $\widetilde{\mathcal{SK}}$, by definition of $\sim^*_{\mathcal{SK}}$ we would need $\mathsf{sk} \in \widetilde{\mathcal{SK}}$ as well and therefore also $\mathsf{sk} \in \mathcal{SK}_{mix}$, a contradiction. Since $\mathsf{pk} \notin \mathcal{PK}_{map}$, by unique provability implied by VRF security we have that therefore $\mathcal{O}_{mix}.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}, x, y, \pi) = 1$.

Now onto verification: Let $b \leftarrow \mathcal{O}_{mix}.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}, x, y, \pi)$ for some $\mathsf{pk} \in \mathcal{PK}_{mix}$.

Using **??** and **??** we can show that for any VF oracles $\mathcal{O}_1, \mathcal{O}_2$, any secret key sk and input x we have that the relevant queries in the combiners evaluation algorithm are in bijective correspondence with the relevant queries in the verification algorithm where a relevant query is defined as follows.

Definition 3.6 (Relevant Oracle Queries)

We formulate the above theorem in the following lemma.

Lemma 3.7 (Relevant Query Correspondence)

Now we initialize the combiner with the oracles $\mathcal{O}_0, \mathcal{O}_{\mathbf{VRF}}$. In a single filtering process the adversary will iterate over all candidate secret keys, and simulate the execution of the combiner by answering any query to $\mathcal{O}_0$ with 0, and for every query in $\mathcal{O}_{\mathbf{VRF}}$ he checks the transcript and answers with an existing query in there if it exists, and otherwise iterates over all possible values (e.g. $\mathcal{Y} \times \mathcal{P}$ if the query is an evaluation) to continue until the output is computed. If for all evaluations the simulated combiner evaluates to $\mathbf{y}$ then we retain the secret key candidate, if for some evaluation the output evaluates to some other value $\mathbf{y}'$ then we discard it.

We argue that a secret key sk is accepted only if the evaluation of the real combiner (with oracle queries) results in the output $\mathbf{y}$. Assume an sk would not evaluate to $\mathbf{y}$ under a candidate sk, then during all evaluations there exists at least one evaluation (the one corresponding to the real secret key) which doesnt evaluate to $\mathbf{y}$. On the same hand for the correct secret key sk the output will always be $\mathbf{y}$, assume that were not the case, that would mean that for some execution we would have needed to guess a subset of oracle queries which did change the output (the 0 guesses to oracle 1 area always right and the lookups in the transcript are also always right). Then however we could simply modify all those outputs under this secret key in the ignored queries to fit that configuration, breaking unique provability. soundness check

Finally since we have shown that the correct secret key will not be thrown out and only secret keys which evaluate to the correct $\mathbf{y}$ are retained we argue that after $2s(\lambda)$ for any uniformly random sample of secret key of the remaining candidates, the probability of sampling a secret key sk' such that on a uniformly random value x' the output diverges from the true output is negligible. The adversary $\mathcal{D}$ can therefore simply do that to break pseudorandomness with high probability. We show pseudocode for $\mathcal{D}$ in **??**.

While robust $(2;1)$ black-box combiners are generally sufficient to construct robust $(n;1)$ black-box combiners, non-existence of a robust $(2;1)$ black-box combiner only allows one to deduce the impossibility of a robust $(n;\lfloor n/2 \rfloor)$ black-box combiner. This holds since we can easily construct a robust $(2;1)$ black-box combiner from a robust $(n;\lfloor n/2 \rfloor)$ black-box combiner by running the robust $(n;\lfloor n/2 \rfloor)$ black-box combiner and for every query made to the first $\lfloor n/2 \rfloor$ oracle candidates we answer with the first oracle candidate given to $\Pi_{\mathsf{VRF}}{}^{(2;1)}$ and for the upper $n - \lfloor n/2 \rfloor$ candidate we simply query the other oracle. If at least one oracle given to $\Pi_{\mathsf{VRF}}{}^{(2;1)}$ implements a secure VRF, then at least $\lfloor n/2 \rfloor$ candidates correspond to a VRF oracle. We can derive the following corollary.

change title to lemma maybe

Lemma 3.8 (Robuts $(n;\lfloor n/2 \rfloor)$ Black-Box VRF Combiner) *There exists no robust $(n;\lfloor n/2 \rfloor)$ black-box combiner in the sense of* **??** *for the VRF primitive defined in* **??** *for all $n \in \mathbb{N}^+$.*

We now aim to extend this result to the weakest form of robust black-box combiners: robust $(n, n-1)$ black-box combiners.

Theorem 3.9 (Robust $(n;n-1)$ Black-Box VRF Combiner) *There exists no robust $(n;n-1)$ black-box* VRF*-combiner.*

Extending the proof of **??** and **??** to this stronger notion requires one main insights: First, repeating the argument before we end up with n oracle queries, of which we can only make 1 trivial (setting it to 0 or similar). The issues is that across oracle calls the individually irrelevant queries may not be jointly irrelevant. That is, changing the outputs of two individually irrelevant oracle queries across two different candidate schemes may result in a different output, making simulation non-trivial.

Proof of ??. TODO

Finally we show that no combiners exist, even if we lower our requirements for the combiner. That is we only require statistical correctness, statistical unique provability and weak pseudorandomness. Even if at least $n-1$ candidates implement secure VRFs, satisfying perfect unique provability, perfect correcntess and strong pseudorandomness. We state the following theorem, the strongest of version of the main result.

Theorem 3.10 (Strong Robust $(n;n-1)$ Black-Box VRF Combiner) *There exists no robust $(n;n-1)$ black-box combiner for the VF primitive satisfying statistical correctness, statistical unique provability and weak pseudorandomness. Even if at least $n-1$ candidate oracles are guaranteed to implement secure VRFs.*

have only VRF here, put the other stuff into corollary

The proof extends the proof of **??**. The weak pseudorandomness we get for free in the previous proof, the constructed adversary can just as well play the

weak prf security experiment and with with the same advantage. To account for the statistical notions of perfect correctness and unique provability we generalize the concept of irrelevance to a probabilistic notion called *overwhelming irrelevance*. We say that a node is *overwhelmingly irrelevant* if the probability, over sampled secret keys $\mathsf{sk} \leftarrow_{\$} \mathsf{Gen}$ and uniformly random inputs $x \leftarrow_{\$} \{0,1\}^{\lambda}$ the probability of the vertex being irrelevant is overwhelming. We claim that for overwhelmingly irrelevant vertices the probability of the verification algorithm requerying is overwhelming. The proof continues with a similar argument, accounting for the fact that $(1 - \mathsf{negl}(\lambda))^{\mathsf{poly}(\lambda)}$ is still overwhelming probability. We provide the entire proof in the appendix.

We briefly want to comment on the combiner notions on other relaxed version of the VRF primitive. That is verifiable unpredictable functions (VUF) as defined in **??**. We note that for unpredictability the same combiner, as given in the proof of **??** works. Therefore as a corollary to **??** we can formulate a similar result on VUFs instead of wVRFs. Since VRF candidates are automatically VUF candidates, this in particular excludes the possibility of a standard VUF combiner.

Corollary 3.11 (Strong Robust $(n; n-1)$ Black-Box VUF Combiner)

3.3 Weak Robust Black-Box VRF-Combiner

Proof (??) Fix a robust $(n; n-1)$ black-box DAG VRF-combiner Π_{VRF} and consider the directed acyclic graph $\mathcal{G}_{eval}$ corresponding to the evaluation algorithm (see **??**). Additionally fix a secret key $\mathsf{sk} \in \mathcal{SK}$ and $x \in \{0,1\}^{\lambda}$. We describe the pseudorandomness adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$ as follows: $\mathcal{A}_1$ samples $2s(\lambda)$ many evaluation points $x_i \leftarrow_{\$} \{0,1\}^{\lambda}$ uniformly at random and obtain the evaluation from the PRF oracle $\Pi^{\mathbf{O}}_{\mathsf{VRF}}.\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}, \cdot)$ to construct $\{(x_i, y_i, \pi_i)\}_{i \in [2s(\lambda)]}$. For each tuple (x_i, y_i, π_i) the adversary, given the public key pk computes a transcript of the combiners verification algorithm $\mathcal{T}_i$ which contains all oracle queries and responses made as well as the computed functions. Now additionally he computes *default* values $z^{eval}_{i,j} := \mathcal{O}_i.\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}^*_i, x_j)$ for all $i \in [n]$ and $j \in [2s(\lambda)]$ and a $\mathsf{sk}^* \leftarrow_{\$} \mathcal{O}_i.\mathsf{Gen}_{\mathsf{VRF}}(1^{\lambda})$. For each $i = 1, \ldots, 2s(\lambda)$ the adversary $\mathcal{A}_1$ computes $\mathcal{SK}_i$ which is defined as:

$$\mathcal{SK}_i := \left\{\mathsf{sk} \in \mathcal{SK}_{i-1} : \tilde{f}(\mathsf{sk}, x_i, \mathcal{T}_i) = (y_i, \pi_i)\right\} \text{ where } \mathcal{SK}_0 := \mathcal{SK}$$

Where $\tilde{f}(\mathsf{sk}, x_i)$ runs the evaluation algorithm of the combiner and for every query to an oracle $\mathcal{O}_i$ we simulate the response by first checking the transcript $\mathcal{T}_i$ whether an identical query has been made to use, and otherwise use the default value $z^{eval}_{i,j}$. Since $\tilde{f}(\mathsf{sk}, x_i, \mathcal{T}_i)$ doesnt query any oracles, we can compute all of this us§ing the PSPACE complete oracle PSPACE. We now

argue that following the topological ordering of the oracle queries (vertices in $\mathcal{G}^{eval}$), if a query is not exactly repeated in the combiners verification algorithm, and thus available in the transcript, the query itself is *irrelevant* in the sense that if replaced with $z_{i,j}^{eval}$ the evaluation of the combiner doesn't exist. We formally state this result in the following claim:

Claim 3.12 *For a fixed, hidden secret key* sk *and any arbitrary value of* $x \in \{0,1\}^\lambda$ *we have that if an oracle query* $\mathcal{O}_{i,j}(\widetilde{\mathsf{sk}}, \widetilde{x})$ *is done in the evaluation algorithm* $\Pi^{\mathsf{O}}_{\mathsf{VRF}}.\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}, x) = (y, \pi)$ *then the query is either exactly repeated in the evaluation of the combiners verification algorithm on the corresponding public key and* y, π*:* $\Pi^{\mathsf{O}}_{\mathsf{VRF}}.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}, x, y, \pi)$ *or the value of* y *stays the same even if the output of* $\mathcal{O}_{i,j}(\widetilde{\mathsf{sk}}, \widetilde{x})$ *is replaced with any value* $z_{i,j}^{eval}$ *that is in the support of* $\mathcal{O}_{i,j}$*, that is if the query made to* $\mathcal{O}_{i,j}$ *is irrelevant.*

To prove **??** we assume towards a contradiction, that the query to $\mathcal{O}_{i,j}(\widetilde{\mathsf{sk}}, \widetilde{x})$ is neither repeated in $\mathcal{T}_i$, nor irrelevant to the output y. This means that there exist two possible outputs of oracle $\mathcal{O}_{i,j}(\widetilde{\mathsf{sk}}, \widetilde{x})$ that result in different evaluations $y^{(0)} \neq y^{(1)}$. This implies that the combiner either is not perfectly correct or uniquely provable and must either accept both $y^{(0)}, y^{(1)}$ or neither. By perfect correctness we have that $y^{(0)}$ must correctly verify in the real world and $y^{(1)}$ must correctly verify in a world where the oracle $\mathcal{O}_{i,j}$ output is remapped to the output resulting in the evaluation of $y^{(1)}$. Since however the oracle are never queried on the only different remapping of $\mathcal{O}_{i,j}$ both must be accepted in the real world, breaking unique provability.

Using the result of **??** we know that all queries we make that we cannot look up in the verifiers transcript are irrelevant in nature, that is we can replace a single one of them with a default value without changing the combiner evaluation. During evaluation we thus modify the first *unknown* oracle evaluation, i.e. the evaluation not in the transcript. During evaluation we replace it with a default value, once we reach the next unknown oracle evaluation.

For all oracles queries of a single oracle in $[n]$ we have that all queries made that are not repeated in the verification algorithm are as a group irrelevant for the output, that means that any of them can be overwitten / reprogrammed. This cannot be done over multiple oracles however

- Given a secret key sk of an oracle $\mathcal{O}_i$ it is still impossible for an efficient algorithm to find a secret key sk' such that $\mathsf{sk} \sim \mathsf{sk}'$ with non-negligible probability. That means that except with non-negligible probability we dont query an oracle on $\mathsf{sk}_0, \mathsf{sk}_1$ such $\mathsf{sk}_0 \sim \mathsf{sk}_1$ in one execution of the evaluation algorithm.
- We can modify the output of one candidate to break unique provability, perfect correctness and pseudorandomness. If we assume that the

individually irrelevant are not *jointly irrelevant* then by defaulting the oracle output in topological ordering we necessarily arrive at an oracle query q for which defaulting changes the output. We **claim** that this output must therefore also be stored in the transcript of the verification, this is because we can simply rewire the previous oracles to assume the modified outputs to obtain $\mathcal{O}_1^*, \ldots, \mathcal{O}_n^*$ for which the evaluation of the verification algorithm doesnt change, BUT the evaluation of the combiner now would have a relevant and public query not being requeried by the verification.

Note that for rewiring we need to make sure that the oracles $\mathcal{O}_1^*, \ldots, \mathcal{O}_n^*$ are still pseudorandom, uniquely provable and correct, i.e. implement secure VRFs. For all calls to eval we need to remap all calls to eval under *related* secret keys $\mathsf{sk}^* \sim \mathsf{sk}$ and all corresponding verification queries. For a verification query we need to rewire the corresponding evaluation call, the evaluation calls to all related secret keys $\mathsf{sk}^* \sim \mathsf{sk}$ and all corresponding verification queries.

Now assume that we are at an evaluation query to rewire, the only way that the rewiring can impract the output of the combiners evaluation is if there is a *relevant* query to verify with an associated public key on the original oracle output y or if there is a relevant evaluation on a related secret key on that same input. If the latter were the case we could break related key recovery using the combiner. If the former is the case then the verification algorithm should come after the evaluation algorithm since otherwise the evaluation combiner algorithm can predict the y. If it comes later then we can choose this oracle to be the malleable one, modify its output in the eval algorithm, forcing the verification to 0 at the same time TODO

- We claim that if the irrelevant queries are oracle-wide irrelevant, then they are overall also jointly irrelevant. Intuitively this must be the case since, if the oracles were not jointly irrelevant then one oracles irrelevant queries would only be irrelevant for a evaluations of another oracle, and relevant for another specific construction (if that were the case and the other construction would be a secure VRF oracle, then relevant the verification combiner needs to repeat those queries). Since neither that oracle, nor the combiner construction, however has knowledge of how that other oracle evaluates this can clearly not be possible. This would indeed only be possible if the combiner could limit those executions to not secure VRF oracles.

Now we just need to somehow get rid of the irrelevant oracle calls and replace them with a deterministic algorithm to do the key-filtering... How about a function $f(x, y)$ such that for a fixed $\hat{x}, \hat{y}$ we have $f(\hat{x}, y_0) = f(\hat{x}, y_1)$ and $f(x_0, \hat{y}) = f(x_1, \hat{y})$ for all y_0, y_1, x_0, x_1 but $f(x_0, y_0) \neq f(x_1, y_1)$.

One example is given by $f(x, y) = \max\{x, y\}$.

We now iterate over the oracles in topological ordering. For every query we precompute a default value which is equivalent to the corresponding oracle query on a random input, either (sk, x) or (pk, x, y, π) where sk are tested to not be *related*. Now we iterate over the oracle queries in topological ordering, for each query in the transcript we answer with the correct answer, for an answer in the cache we answer with the answer, for every query not in the transcript we default and store the mapping in the cache. Assume at step v_i the combiner output changes, as opposed to if we continue computation with the oracles, then we know that the query was not in the transcript (otherwise the execution would be the same). Since the query is not in the transcript and the query changes the combiner output this must mean that the combiner is necessarily not uniquely provable or perfectly correct.

We now aim to show that the combiner, however, must be uniquely provable and perfectly correct, since we can find VRF secure instanciation of the oracles that align with the evaluation transcript and the subsequent honest evaluation. Assume towards a contradiction that at least two candidates have no *VRF-secure extension*, that means that they cannot be extended from the existing mappings in a way such that at least $n - 1$ of them are VRF secure. That means that there must exist a conflict, we enumerate all possible conflicts at every time step i, assuming the prior modifications left the cnadidates all VRF secure (base case $i = 0$ where n are assumed to be secure)

- Assume we encounter an evaluation query with input (sk, x) and default it to (y, π). The only way that this would break unique provability or perfect correctness would be if there exists a related sk' which outputted $(y' \neq y, \pi)$. Assuming that related keys cannot efficiently be found this case only holds with negligible probability. Otherwise there exists a prior verification query which evaluated to $(\mathsf{pk}, x, y, \pi) \mapsto 0$ for pk associated with sk then we simply make this oracle the insecure one. If on the other hand there exists a verification query to $(\mathsf{pk},)$ problem
- Assume we encounter a verification query with input (pk, x, y, π) then this can only break a candidates unique provability or perfect correctness if there exists a verification on 1 (must be checked) on a related public key, we assume that also finding related public keys is hard. Or there must exist an evaluation on a related secret key sk, x that results in y, π with sk associated to pk. If that also was in the transcript that doesnt work, since we assume all candidates secure then. If the evaluation was defaulted the probability of that happening
- A query we defaulted was queried on eval for (sk', x) to (y', π') even though there is another query to (y, π) on (sk, x). In that case if $\mathsf{sk} \sim \mathsf{sk}'$ and $y' \neq y$ we have a contradiction to unique provability. However

note that $\mathsf{sk} \sim \mathsf{sk}'$ requires the combiner to come up with two related secret keys, which we figure is impossible for efficient algorithms.

- A query we defaulted was queried on eval for (sk', x) to (y', π') even though there is another query to verify on $(\mathsf{pk}, x, y', \pi') = 0$ or on $(\mathsf{pk}, x, \tilde{y} \neq y', \tilde{\pi}) = 1$. We default all verification queries to 0 and exclude answeres on y' in simulation that we have seen. Either the evaluation is checked and requires evaluation to y' and we answer the verification with 0,

  Either the verification query evaluated to 1 by being in the transcript, then the output needed to be computed

Note that for all previous queries there is no conflict, either the queries are done to evaluation and there we **assume** that the evaluation algorithm cannot find two related queries, hance the default answeres just redefine outputs of the equivalence classes of related secret keys. Verification queries are answered with 1 on (pk, x, y, π) if and only if for all replays with a different choice of y of queries answered with y

If an oracle query cannot be found in the transcript or the cache we evaluate it We now want to show that if oracle-wide irrelevant queries can be remapped without changing the combiners evaluation, overall oracles the irrelevant queries are jointly irrelevant as well. We proceed with the following argument: By showing that we can remap those outputs in a way such that there exists an n-tuple of secure VRF oracles that matches this evaluation transcript in topological ordering of the irrelevant vertices, then if we encounter a vertex which becomes relevant through that remapping this query must necessarily be repeated

—

Assume, for a warm-up that $n = 2$. If we know that at least one candidate is VRF secure then we know that during the execution of evaluation of the combiner on (sk, x) any relevant query made by either oracle must be re-queried by the verification algorithm. If that were not the case then simply let this be the malleable oracle, then produce two outputs, which by perfect correctness both must verify, breaking unique provability.

Since this has to hold for both oracles, just initialize one of the two oracles with the malleable one, this execution becomes simulatable entirely. The other execution is given in parts by the transcript of the verification algorithm where the entries that are not contained in the transcript can simply be simulated with random evaluations.

—

what happens if the query input to one that was previously in the transcript changes as a result of the wrong but output-preserving guess of an irrelevant oracle. I think it will be the case that for any new transcript pro-

Warm up. Lets first consider the case $n = 2$. We consider two cases: Case 1 is that for all uniquely provable oracles $\mathcal{O}_1, \mathcal{O}_2$, all secret keys $\mathsf{sk} \leftarrow_\$ \mathsf{Gen}_{\mathsf{VRF}}$ and all $x \leftarrow_\$ \{0,1\}^\lambda$ we all relevant oracle queries must are repeated in the transcript of the verification algorithm. Assume this were not the case, that is, a relevant oracle query for say $\mathcal{O}_1$ would not be re-queried in the verification, then we could obtain two verifiying distinct images $y_0 \neq y_1$ by using correctness but breaking unique provability. Since, in particular, we can make the oracles uniquely provable, , we solve this case. unsure

We want to mention that instead of achieving weak pseudorandomness we can also allow unpredictability instead. As a corollary we can exclude the possibility of wVRF combiners in the sense of **??** and VUF combiners in the sense of **??**.

Corollary 3.13 (Impossibility of a Robust $(n; n-1)$ Black-Box VRF-combiner) *Given n secure VRF candidate schemes $\mathcal{O}_1, \ldots, \mathcal{O}_n$ there exists no robust black-box combiner that implements a secure VRF scheme if even one candidate scheme is not uniquely provable.*

Corollary 3.14 (Impossibility of a Robust $(n; n-1)$ Black-Box VUF-combiner) *Given n secure VUF candidate schemes $\mathcal{O}_1, \ldots, \mathcal{O}_n$ there exists no robust black-box combiner that implements a secure VUF scheme if even one candidate scheme is not uniquely provable.*

Having ruled out the possibility of combining VRFs, VUFs, wVRFs even for statistical notions of correctness and unique provability we need to make an even stronger assumption to allow for combiners to exist. We will assume that all candidates satisfy unique provability. While this might sound like a very strong assumption, which it is, we argue that it is not too hard of a restriction on modern schemes that pose potential candidate schemes for a combiner. This is because unique provability is an information theoretic notion which is typically mathematically proven for good constructions such as ECVRF or Lattice VRF.

However, assuming that all candidates satisfy unique provability combiner constructions become very simple. We present a construction very similar to the simple xor combiner for PRFs that works as follows. The key generation algorithm is returns a tuple of n secret keys and a tuple of n public keys, where the i'th entry is an independently sampled key from candidate i, the evaluation algorithm evaluates x once for each candidate under the corresponding secret key and XORs the results together to obtain the image, additionally the proof is simply the transcript of the execution. The verification algorithm checks whether all oracle calls align by iterating over the queries made in the transcript and verifying each with a call to the corresponding verification oracle with the corresponding public key. Afterwards we recompute y by xoring the partial outputs of the candidates to

verify that we obtain the same image. To prevent the combiner from being incorrect if a candidate it incorrect we execute a public checking routine before evaluating any algorithm. The public checking routine consists of sampling a polynomial number (in λ) of secret keys sk and evaluation points x, computing their evaluation (y, π) and then verifying with the corresponding public key pk for each oracle. If one oracle behaves incorrectly it is ignored during the execution of the combiner. Allowing the combiner to achieve statistcal correctness. Note that this combiner already works if only at least one candidate is secure. We state this result as a lemma below and show the construction of the combiner in **??**.

DOESNT WORK DUE TO UNIQUE PROVABILITY BREAK when collision is found but not remarked Here couldn't the evaluation simply involve a check to verify. If this check fails then a candidate fails, this is proven to the verification algorithm by providing an incorrect evaluation/verification in the proof. If the verifier can detect that fault then he accepts the skipping of the candidate, if he doesn't detect the fault then he cannot accept the skipping and rejects if that flag is set or if the other verification doesnt accept.

Algorithm $\Pi^{\mathcal{O}_1,\mathcal{O}_2}_{\mathsf{VRF}}.\mathsf{Gen}_{\mathsf{VRF}}(1^\lambda)$

1 : $(\mathsf{sk}_1, \mathsf{pk}_1) \leftarrow_\$ \mathcal{O}_1.\mathsf{Gen}_{\mathsf{VRF}}(1^\lambda)$
2 : $(\mathsf{sk}_2, \mathsf{pk}_2) \leftarrow_\$ \mathcal{O}_2.\mathsf{Gen}_{\mathsf{VRF}}(1^\lambda)$
3 : **return** $((\mathsf{sk}_1, \mathsf{sk}_2), (\mathsf{pk}_1, \mathsf{pk}_2))$

Algorithm $\Pi^{\mathcal{O}_1,\mathcal{O}_2}_{\mathsf{VRF}}.\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}, x)$

1 : $(\mathsf{sk}_1, \mathsf{sk}_2) \leftarrow \mathsf{sk}$
2 : $(y_1, \pi_1) \leftarrow \mathcal{O}_1.\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}_1, x)$
3 : $(y_2, \pi_2) \leftarrow \mathcal{O}_2.\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}_2, x)$
4 : $y \leftarrow y_1 \oplus y_2$
5 : $\pi \leftarrow (y_1, \pi_1, y_2, \pi_2)$
6 : **return** (y, π)

Algorithm $\Pi^{\mathcal{O}_1,\mathcal{O}_2}_{\mathsf{VRF}}.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}, x, y, \pi)$

1 : $(\mathsf{pk}_1, \mathsf{pk}_2) \leftarrow \mathsf{pk}$
2 : $(y_1, \pi_1, y_2, \pi_2) \leftarrow \pi$
3 : **req** $y = y_1 \oplus y_2$
4 : **req** $\mathcal{O}_1.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}_1, x, y_1, \pi_1) = 1$
5 : **req** $\mathcal{O}_2.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}_2, x, y_2, \pi_2) = 1$

Figure 3.3: Combiner for VRFs if all candidate schemes are uniquely provable.

Lemma 3.15 *There exist a robust $(n;1)$ black-box combiner that combines n uniquely provable VF candidates, of which at least one implements a secure VRF to construct a VF that is statistically correct in the sense of* **??**, *uniquely provable in the sense of* **??** *and pseudorandom in the sense of* **??**.

Intuition. The proof of the security works in two stages. First we show that by doing the public correctness check we can detect incorrect candidates with overwhelming probability. Then we show that given any adversary $\mathcal{A}$ against the pseudorandomness of the combined scheme we can construct a reduction $\mathcal{R}$ against the pseudorandomness of any underlying candidate. The reduction works by simulating all other candidates faithfully, relying on the oracle in the prf security experiment to simulate the attacked candidate, and then use $\mathcal{A}$. Unique provability follows from unique provability of all underlying candidates.

Proof (??)

should be easy

Having shown that general notion of combiners on VRFs even with relaxed requirements on the security properties of the combiner are impossibile and the combiners on VRFs with guaranteed unique provability are easy we want to strengthen the result by considering combiners for VRFs (assuming all candidates are uniquely provable) with advanced additional properties, namely constrainability as in **??**, key-homomorphism as in **??**, aggregability as in **??** and finally unbiasability as in **??**.

For the notions of constrainability, key-homomorphism and aggregability this question is quckly answered as we we show how the above construction of extended PRF combiner can be minimally adapted to combine the notions. We will quickly explain how to extend the construction of the extended PRF combiner to capture those notions and then we will move on to the more interesting notion of unbiasability, for which the constructions become more complex.

We now consider additional security properties for VRFs, aggregatability that allows the aggregation of many secret keys or image-proof pairs to allow for efficient batch verification and unbiasability which prevents adversaries from skewing the output distribution of a VRF by maliciously selecting the keys.

Robust Black-Box Combiner for Unbiasable VRF. We present a robust black-box combiner for unbiasable VRF under the assumption that all candidate schemes are uniquely provable. We first note that there exists a compiler to reduce unbiasable VRF to VRF under computational Diffie-Hellman (CDH) assumption [**?**], a combiner that works under the CDH assumption would be to ignore unbiasability of the candidates and combining normal VRFs and then introducing unbiasability afterwards . Instead of relying on an additional hardness assumption we prefer to use the unbiasability property of the secure candidates. We introduce a robust $(2;1)$ black-box combiner for unbiasable VRF based on collision resistant hash functions and randomness extractors that is heavily inspired by the construction of unbiasable VRF presented in [**?**]. The construction works by initially splitting the input

is this even a robust black-box combiner since we cannot reduce unbiasability from the combiner to its candidates.

$x \in \{0,1\}^\lambda$ into two parts $x = x_0 \| x_1$ then hashing $x_1 \| 0$ and $x_1 \| 1$ using a collision resistant hash function. Finally we evaluate each of the candidate on one point $H(x_1 \| 0)$ and $H(x_1 \| 1)$ before running a randomness extractor to extract randomness from x_0 and adding it to $y_0 \oplus y_1$. We formulate the result formally as a theorem.

Theorem 3.16 (Robust $(2;1)$ Black-Box Combiner for Unbiasabile VRF) *There exists a uniform robust $(2;1)$ black-box combiner for unbiasable VRF if all candidate schemes are uniquely provable.*

Robust Black-Box Combiner for Aggregatable VRF. We first look at key-homomorphism combiners. Given a key-homomorphic function given an evaluation point $x \in \{0,1\}^\lambda$ we show how to publicly check for the homomorphism property of each candidate scheme. This is possible due to an algorithm by Blum **[?]** which introduces an efficient homomorphism check that runs in time independent of the group size. This means that given an evaluation point we can check whether the evaluation function is a group homomorphism with respect to the secret-key argument with high probability in time independent of the size of the secret-key space. We state this claim in form of a lemma and quickly elaborate on the homomorphism check, referring to the original result for the complete proof.

Lemma 3.17 (Robust $(2;1)$ Black-Box Combiner for Key-homomorphic VRF)

Next we consider aggregable VRFs. We show that indeed there exists a combiner

3.4 Discussion and Extensions

There are different ways to combine VRFs, for both unique provability and key-homomorphism they cannot be combined in a black-box fashion if we want to preserve properties such as unpredictability and therefore also randomness. On the other hand we manage to construct a robust black-box combiner that combines unbiasabile VRFs under guaranteed unique provability for all candidates. We find that the black-box combination of information theoretic properties generally seems hard if other properties such as unpredictability need to be preserved. While the class of all robust black-box combiners initially seems to be very broad we show that we can capture all polynomially bounded combiners using the notion of a robust black-box DAG combiner, which allows a more systematic analysis of possible constructions by abstracting a graph structure in the computation executed by the implemented algorithms. We have not considered possible combiner constructions for stronger relaxations, such as computational unique provability

or correctness with error, this is mainly because the resulting construction only offer weak guarantees that are insufficient for practical applications.

Chapter 4

Conclusion

Appendix

Proof of ??

Proof It is sufficient to find a $(1;2)$-robust black-box combiner for VRF where all candidates are uniquely provable. Given a combiner we can then construct a $(1;n)$-robust black-box combiner for VRF by nesting the combiners iteratively. A construction for the combiner is given in **??**. The key generation works by querying both candidates on the given security parameter and grouping the secret and public keys respectively. Evaluation again queries both candidate schemes on the input with their respective secret key and returns the xor of the outputs while the the individual outputs and proofs consistute the new proof. The verification algorithm checks whether the image y is the result of xoring the individual outputs of the candidates and whether the individual images y_1, y_2 verify with respect to the verification key the proof and the pre-image. To ensure that the combiner is still efficient we sim

The combiner is clearly efficient since every algorithm consists of one call to the respective algorithm of each candidate scheme (whi)

Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Declaration of originality

The signed declaration of originality is a component of every written paper or thesis authored during the course of studies. **In consultation with the supervisor**, one of the following two options must be selected:

☐ I hereby declare that I authored the work in question independently, i.e. that no one helped me to author it. Suggestions from the supervisor regarding language and content are excepted. I used no generative artificial intelligence technologies[1].

☐ I hereby declare that I authored the work in question independently. In doing so I only used the authorised aids, which included suggestions from the supervisor regarding language and content and generative artificial intelligence technologies. The use of the latter and the respective source declarations proceeded in consultation with the supervisor.

Title of paper or thesis:

Authored by:
If the work was compiled in a group, the names of all authors are required.

Last name(s):	**First name(s):**

With my signature I confirm the following:
- I have adhered to the rules set out in the Citation Guidelines.
- I have documented all methods, data and processes truthfully and fully.
- I have mentioned all persons who were significant facilitators of the work.

I am aware that the work may be screened electronically for originality.

Place, date	**Signature(s)**

If the work was compiled in a group, the names of all authors are required. Through their signatures they vouch jointly for the entire content of the written work.

[1] For further information please consult the ETH Zurich websites, e.g. https://ethz.ch/en/the-eth-zurich/education/ai-in-education.html and https://library.ethz.ch/en/researching-and-publishing/scientific-writing-at-eth-zurich.html (subject to change).